

Manual Testing — Complete Study Notes

Sprint & Release Process + Black-Box Test Design Techniques

PART 1: SPRINT vs RELEASE (Foundation)

1.1 Why Sprints Exist

A big application (e.g., an online shopping app) has many features: Login, Signup, Add to Cart, Payment, Order Tracking, Wishlist.

Developers can't build all of this in one day — so the work is broken into **Sprints**. After enough Sprints are done, the working software is given to customers — that event is called a **Release**.

Term	Simple Meaning
------	----------------

Sprint	A short, fixed time-box (usually 2 weeks) where the team builds, tests, and completes a small set of features.
---------------	--

Release	Delivering a stable, tested version of the software to real customers.
----------------	--

Analogy: Writing a book — each finished chapter = a Sprint. Publishing the whole book = a Release.

1.2 Sprint Duration

Usually 1, 2 (most common), 3, or 4 weeks.

1.3 Sprint Flow

Product Backlog → Sprint Planning → Development → Testing
→ Bug Fixing → Retesting → Sprint Review → Sprint Retrospective → Next Sprint

1.4 Sprint Phases Explained

Phase	What Happens
Product Backlog	Product Owner lists <i>all</i> features needed (Login, Signup, Search, Payment, Cart, Profile...)
Sprint Planning	Team decides <i>what</i> will be done this sprint, <i>who</i> does it, and <i>how long</i> it takes
Development	Developers write code for assigned features
Testing	Testers execute test cases on the new features
Bug Reporting	If something fails, a bug is logged (ID, Expected, Actual, Priority, Severity)
Bug Fixing	Developer fixes the reported bug
Retesting	Tester re-checks that <i>specific</i> bug — Pass or Reopen
Sprint Review	Team demos completed work to Product Owner/client
Sprint Retrospective	Team discusses what went well / wrong / how to improve

1.5 Sample Bug Report Format

```
Bug ID      : BUG-101
Feature     : Login
Expected    : Login should open dashboard
Actual      : Application crashes
Priority     : High
Severity    : Critical
```

- **Priority** = how urgently it should be fixed (business urgency)
- **Severity** = how badly it affects the application (technical impact)

1.6 Release Flow

```
Many Sprints Completed → Final Testing → Regression Testing
→ User Acceptance Testing (UAT) → Approval → Deployment
→ Release to Users → Maintenance
```

Step	Meaning
Final Testing	Test the whole app together (Login, Payment, Search, Logout, Orders...)
Regression Testing	Confirm new changes haven't broken old features
UAT	Client/business users test it against real-world use
Deployment	Developers/DevOps push the build to the production server
Release	Customers can now use the new version
Maintenance	Ongoing bug fixes, performance improvements, new features

1.7 Sprint vs Release — Quick Comparison

Sprint	Release
Small development cycle	Final software delivery
Usually 1-4 weeks	Happens after one or more sprints
Builds a few features	Delivers many completed features
Internal team activity	Delivered to customers
Can contain bugs fixed within the sprint	Should be stable & production-ready
Ends with Review + Retrospective	Ends with deployment to users

Key Interview Point: Every sprint produces working software, but **not every sprint is released**. Several sprints are usually combined into a single release.

PART 2: REAL INDUSTRY RELEASE WORKFLOW (Detailed)

2.1 Full End-to-End Flow

Customer Requirement

↓

BA Creates Requirements (BRD/SRS)

↓

Requirement Analysis

↓

Sprint Planning

↓

Task Assignment

↓

Developer Codes (3-Layer Architecture)

↓

Code Review

↓

Git Commit (GitHub/GitLab/Bitbucket/Azure DevOps)

↓

CI/CD / DevOps Creates Build (ZIP/WAR/JAR/APK/EXE/Docker)

↓

Deploy to QA/Test Server

↓

Smoke Testing

↓

Functional Testing

↓

Bug Found? → Bug Report in Jira

↓

Team Lead Reviews & Assigns Bug

↓

Developer Fixes Bug → Commits Code

↓

New Build Generated & Deployed

↓

Retesting (of that bug only)

↓

Regression Testing (whole affected area)

↓

Sprint Review → Sprint Retrospective

↓

Next Sprint Starts (in parallel with testing of previous sprint)

↓

After Multiple Sprints → Final Regression → UAT

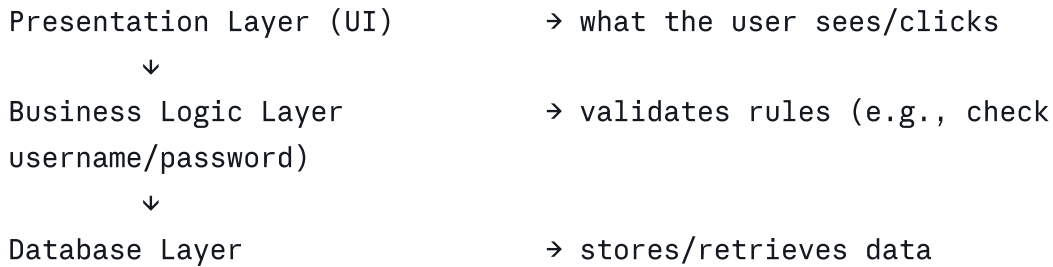
↓

Release Approval → Production Deployment



Live for Customers → Maintenance

2.2 3-Layer (3-Tier) Architecture



Example: User clicks Login (UI) → system checks credentials (Business Logic) → reads user record (Database) → Dashboard opens.

2.3 How Code Reaches Testers — Two Realities

Modern companies (most common today):

Developer → Git Commit → CI/CD Pipeline (Jenkins/GitHub Actions/Azure DevOps) → Automated Build → Auto-Deploy to QA server → Tester tests via a URL

No manual ZIP sharing — everything is automated.

Small/legacy companies (still happens sometimes):

Developer → Build → ZIP/WAR/JAR/APK/EXE → Shared Folder/Email → Tester downloads & installs

Interview-ready answer: "In modern companies, developers commit to Git and DevOps/CI-CD automatically builds and deploys to a test server — testers access it there. In some smaller or legacy setups, a ZIP/WAR/JAR/APK/EXE file is shared manually for testers to install."

2.4 Smoke Testing vs Functional vs Regression

Type	Purpose	When
Smoke Testing	Quick check — does the build even work? (app opens, login works, main pages load)	Immediately after a new build is deployed
Functional Testing	Detailed testing of each feature against requirements	After smoke test passes
Regression Testing	Confirm nothing <i>old</i> broke because of a <i>new</i> change	After any bug fix or new feature

If Smoke Testing fails → build is **rejected immediately**, no further testing happens until a new build is given.

2.5 Parallel Work Reality

Testers testing Sprint 1 ≠ Developers waiting idle. Developers usually **start Sprint 2 immediately**, so testing and development run in parallel across sprints — this is what makes Agile fast.

PART 3: BLACK-BOX / SPECIFICATION-BASED TESTING TECHNIQUES

These are **test design techniques** — methods to decide *which* test cases to write, without looking at the code (black-box). This is what you do during "Functional Testing" (see Part 2.4) — these techniques tell you *how* to pick smart test cases instead of guessing.

3.1 What is Specification-Based Testing?

Definition: Specification-based testing (a.k.a. black-box testing / behavioral testing) means designing test cases purely from the **requirements/specification document (SRS/BRD)** — you never look at the internal code. You only care about: *given this input, what output should the system produce?*

Why it matters (interview angle): Manual testers rarely see source code. All testing is based on what the requirement document says the system *should* do — so specification-based techniques are the core toolkit of manual testing.

Common Specification-Based Techniques:

1. Equivalence Partitioning (EP)
2. Boundary Value Analysis (BVA) — natural partner to EP
3. Decision Table Testing
4. State Transition Testing
5. Use Case Testing

We'll go deep into the three you asked for: **Equivalence Partitioning** and **Decision Table Testing** (plus BVA, since it's always taught alongside EP).

3.2 Equivalence Partitioning (EP)

Concept

Instead of testing *every possible input* (impossible — infinite combinations), divide inputs into groups ("partitions" or "classes") where **all values in a group should behave the same way**. Then test **just one representative value from each group** — if one value in the group works, you assume the rest do too.

Why It Saves Time

If a Login field accepts passwords of length 8-15 characters:

- Testing every length (1 char, 2 chars, 3 chars ... 100 chars) = hundreds of test cases
- EP says: group them into **Valid** (8-15) and **Invalid** (<8 or >15) partitions, and test just 1 value from each.

Types of Partitions

- **Valid Equivalence Class** — inputs the system should accept
- **Invalid Equivalence Class** — inputs the system should reject

Example — Login Password Field (Rule: must be 8-15 characters)

Partition	Range	Example Test Value	Expected Result
Invalid (too short)	< 8 chars	"abc123" (6 chars)	Rejected
Valid	8-15 chars	"pass1234" (8 chars)	Accepted
Invalid (too long)	> 15 chars	"thispasswordis16c" (17 chars)	Rejected

Just **3 test cases** cover the entire input range instead of testing every possible length.

Example — Age Field (Rule: Accepts age 18 to 60)

Partition	Test Value	Expected
Invalid (below range)	10	Rejected
Valid (in range)	30	Accepted
Invalid (above range)	70	Rejected
Invalid (non-numeric)	"abc"	Rejected

Example — Non-Numeric Input Type

Username field, must contain only letters:

Partition	Example	Expected
Valid	"Apeksha"	Accepted
Invalid — contains numbers	"Apek123"	Rejected
Invalid — contains special chars	"Apek@#"	Rejected
Invalid — empty	""	Rejected

Key Interview Point

"EP reduces the *number* of test cases while still covering all possible behaviors of the system, by grouping inputs that are expected to be treated the same way."

3.3 Boundary Value Analysis (BVA) — Natural Partner to EP

Concept

Most bugs happen **at the edges** of a valid range, not in the middle. So instead of (or in addition to) EP, test the exact **boundary values** — the min, min-1, min+1, max, max-1, max+1.

Example — Same Age Field (Valid range: 18-60)

Test Value	Type	Expected
17	Just below min	Rejected
18	Min boundary	Accepted
19	Just above min	Accepted
59	Just below max	Accepted
60	Max boundary	Accepted
61	Just above max	Rejected

Rule of thumb: EP tells you *which groups* to test; BVA tells you *exactly where* within/around those groups bugs are most likely hiding. In real projects, testers combine both — EP for broad coverage, BVA for edge precision.

3.4 Decision Table Testing

Concept

Used when the system's output depends on **multiple input conditions combined** — i.e., business logic with several rules (if this AND that, then this happens). A Decision Table lists every combination of conditions and the expected action for each combination, so no combination is missed.

When to Use It

- Login with multiple conditions (username valid? password valid? account locked?)
- Discount rules (is member? order value > 1000? coupon applied?)
- Loan eligibility, insurance claim rules, form validations with dependent fields

Structure of a Decision Table

Conditions (Rules) → Rule 1 | Rule 2 | Rule 3 | Rule 4 ...
Actions/Outcomes → What happens in each rule

Example — E-commerce Discount Rule

Requirement:

- If the customer is a **Member** AND order value > ₹1000 → 20% discount
- If Member but order value ≤ ₹1000 → 10% discount
- If Not a Member but order value > ₹1000 → 5% discount
- If Not a Member and order value ≤ ₹1000 → No discount

Condition	Rule 1	Rule 2	Rule 3	Rule 4
Is Member?	Yes	Yes	No	No
Order Value > ₹1000?	Yes	No	Yes	No
Action → Discount	20%	10%	5%	0%

Each **column** becomes exactly **one test case**:

Test Case	Is Member	Order > ₹1000	Expected Discount
TC1	Yes	Yes	20%
TC2	Yes	No	10%
TC3	No	Yes	5%
TC4	No	No	0%

Example — Login with Account Lock Rule

Requirement:

- Valid username + Valid password + Account not locked → Login success
- Valid username + Valid password + Account locked → "Account Locked" message
- Invalid username or invalid password → "Invalid Credentials" message

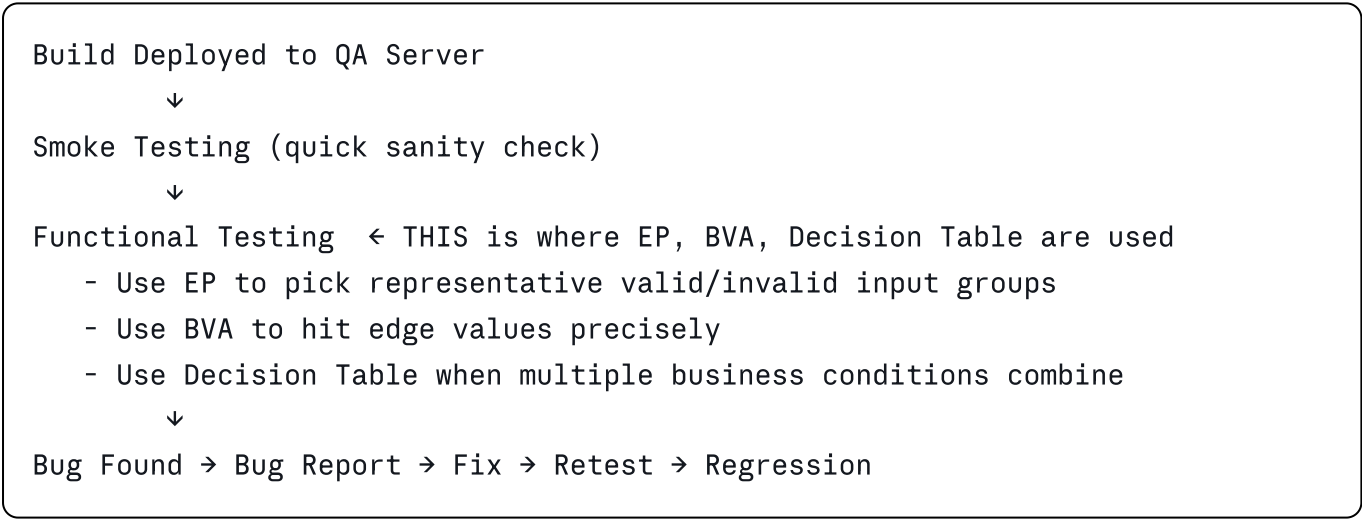
Condition	Rule 1	Rule 2	Rule 3
Username Valid?	Yes	Yes	No
Password Valid?	Yes	Yes	Yes/No
Account Locked?	No	Yes	—
Action	Login Success	Account Locked Msg	Invalid Credentials Msg

Why Decision Tables Matter (Interview Point)

"Decision Table testing ensures **no combination of business rules is missed**. It's especially useful when conditions are interdependent — where testing them one at a time (like plain EP) wouldn't reveal a bug that only appears when two or more conditions interact."

3.5 How These Techniques Fit Into the Sprint/Release Flow You Learned

Recall from Part 2 — after a build is deployed to the QA server, the tester does **Functional Testing** (Step 12/13 in the workflow). This is exactly where these techniques are applied:



Quick Summary Table

Technique	Best Used When	Goal
Equivalence Partitioning	Single input field with a range/category of valid values	Reduce test cases by grouping similar inputs
Boundary Value Analysis	Same as above, but focus on edges	Catch bugs at exact min/max boundaries
Decision Table Testing	Multiple conditions combine to decide the outcome	Ensure every combination of business rules is covered

PART 4: QUICK INTERVIEW Q&A RECAP

Q1. What is a Sprint? A fixed time period (usually 2 weeks) where the team develops, tests, and completes a set of planned features.

Q2. What is a Release? Deployment of a stable, tested version of the software to end users after one or more sprints.

Q3. Does every sprint result in a release? No — every sprint produces working software, but releases usually bundle multiple sprints together.

Q4. What is Equivalence Partitioning? A technique that divides input data into groups (valid/invalid) so testing one value per group is enough to represent the whole group's behavior.

Q5. What is Boundary Value Analysis? A technique focused on testing the exact edges of valid input ranges, since most defects occur at boundaries.

Q6. What is Decision Table Testing? A technique that maps every combination of multiple input conditions to its expected output/action, ensuring no business-rule combination is missed.

Q7. Difference between EP and Decision Table? EP tests **one condition/field at a time** by grouping its values. Decision Table is used when **multiple conditions interact together** to decide the outcome.